Arduino Prototype

ME 2900

Instructor: Lauren Eichacker, PhD

Date of Submission: 10/12/26

Justin Lewis

Lewis.3402

Tuesday Lab 1:45-3:35

Introduction:

The driving factor of this prototype project was to brainstorm and design a functional prototype that paired electrical and mechanical components with an Arduino microcontroller in response to a real world problem. With this motivation, I created the SENTRY, a simple PIN controlled infrared anti-intrusion detection system with audible alarm capabilities. The purpose to was create a simple and afford system for any type of user. While home security systems are already on the market, the commercially available systems are usually paired with expensive equipment, lengthy installation processes, and or subscription services are required for the full extent of the system. SENTRY on the other hand offers a micro scaled alternative that shows the needed functions without the complexity of a commercial monitoring system. The main components of the SENTRY prototype are the Arduino UNO R3, IR distance sensor, 4 by 4 membrane keypad, LED visual indicators, and an active buzzer. The system lets its user activate and deactivate the system using their desired 4 digit PIN, while active, it detects objects that enter the protected area, and sounds off an alarm that is paired with a flashing indicator.

Idea Development Process:

The initial brainstorming process led me to a different idea at first. Initially, the goal was to better the operation of residential garage door safety systems by eliminating the illegitimate detections that many suffer from due to sunlight interfering with the actual infrared sensors. The idea was that a system that could tell the difference between an actual obstruction and sunlight interference, however the complexity of the garage system and the available time and resources led to a different application of IR sensing. The second rendition landed on the IR anti intrusion system as it still demonstrated sensor integration with user control. Originally the prototype was based off of your typical tripwire style alarm system, but for a better user experience and UI, the final rendition involved the addition of a keypad controlled system that utilized a user chosen 4 digit PIN to arm and disarm the system.

Multiple components were brainstormed and tested during the development process. First off, the circuit was designed using a push button to mimic that an intrusion had occurred. This first test run allowed for testing of the basic alarm logic before the IR sensor was integrated. Once the button variation successfully activated the buzzer and indicators, and proved the logic was working, it was replaced with an actual IR distance sensor. This finally allowed for object detection testing without needing to physically touch the hardware. The idea to boost the users experience with the UI was also a thought that was considered, where it was attempted to incorporate an LCD to allowed for the state of the system to be visually displayed to the user. Although this was a good idea, it was ultimately scrapped for the keypad for user authentication

and LED indicators to visually communicate the state of the system to its user. While the flashing red LED indicator does show the user that the alarm has been tripped, it was concluded that this wasn't enough, therefore, an audible alarm (active buzzer) adds an additional level of security for intrusion indication. This final arrangement was selected ultimately because it reduced the complexity of the SENTRY by eliminating components and reducing the amount of connections needed within the main circuit.

Discussion:

The final SENTRY prototype is a keypad governed infrared intrusion detection system that was conceived to observe a specified entryway and notify its user whenever an object may cross into the designated protected area. SENTRY is the culmination of an Arduino UNO R3, 4 by 4 membrane keypad, IR distance sensor, two different colored LEDs, and an active (piezo) buzzer. All of the following components were then integrated within a cardboard casing that is meant to mimic a small scale security installation of a miniature room. When the SENTRY is given power, it's initial state is disarmed with LEDs and the buzzer deactivated. For the user to activate the SENTRY, they must enter a predetermined 4 digit PIN (here it is 1234) using the membrane keypad. After this action is taken, the SENTRY will only activate once the correct PIN is entered, and once it does, the green LED indicator will activate to allow the user to know that the system is armed. However, the system doesn't initialize immediately, there is a 2 second delay to eliminate any false readings like the user activating the system while within the protected area. Once the delay is over, the SENTRY will begin is typically active monitoring. While active, the SENTRY does require that the intruder or object must be within the detection range for at least 250 milliseconds, any longer than that, and the system will assuming an intrusion has occurred. If an intrusion occurs, then the green LED will deactivate, the red LED will flash continuously in 400 millisecond intervals, and the buzzer will activate and provide an audible alarm indication. An important note on the SENTRY; an active alarm state will stay in that state post removal of the detected object. The idea behind this is to prevent the alarm from resetting as soon as the intrusion is no longer detected, requiring user input to deactivate the alarm. For the SENTRY to change from an active alarm state to its initial state, the correct 4 digit PIN must be input again. This idea also works when the system is armed but not tripped, entering the PIN will still deactivate the system, while an incorrect one will be ignored.

The Arduino programming and control logic was developed within Arduino IDE and utilizes the Keypad library to process the PIN that the user inputs to the membrane keypad. The program also allows the user to track the state of the system, and whether or not an intrusion has occured. At the initialization, the general setup function sets the component pins as outputs and inputs, deactivates the alarm indicators, and sets the serial communication in IDE to 9600 baud. The

loop function is then used to perpetually read the IR sensor, while also checking for user input on the keypad. The keypad works because its authentication process will compare the users inputted PIN against the known/stored PIN password, thus if the entered PIN is correct, then the system will decide to arm itself, disarm itself, or reset the active alarm depending on the state of the system at the time. If a user inputs an incorrect password, that entered PIN will be rejected and cleared so that the user may make another attempt. The actual IR detection is governed by looking at the sensor's analog reading with the threshold that was determined during testing, and when a reading is greater than or equal to that threshold, the program will start to track the duration of that detection with the Millis function. Therefore, if a an intrusion lasts for a minimum of 250 milliseconds, then the alarm will be activated, and the user will be notified to eliminate the probability of false readings setting the alarm off. Millis is also integrated in the program to allow for a 2 second delay when arming the system, and to govern the flashing red LED indicator. Multiple user defined functions for integrated into the Arduino program, each used for something different within the program, but ultimately added to make the program easier to understand. The armSystem function will activate the armed state of the SENTRY, while illuminating the green LED, and the disarmSystem will do the opposite, and reset the system and turn off the alarm outputs. The function triggerAlarm will turn the audible and visual alarm indicators to their active state, and finally, the objectDetected function is used to take the sensor reading as an argument and then return a Boolean value that shows whether or not the measured reading is within the detection threshold. These functions increased the readability of the sketch and made troubleshooting much easier.

Conclusion:

The creation of SENTRY was considered a success as it showed the combination of integrated IR sensing, PIN authentication, and automatic alarm control could be done utilizing an Arduino microcontroller. SENTRY also met the design objectives for a simple UI that allowed for arm/disarm capabilities and easy to understand visual and audible indications. The main issue or difficulty that was encountered was the calibration of the IR distance sensor. SENTRY needed a reliable object detection device for the determined range, but initial testing showed that the sensor's analog readings varied by a large margin depending on the distance of the object which made it increasingly difficult to determine the actual detection threshold. There were times where the alarm would activate as soon as the SENTRY was armed. To work around these problems, the sensor's reading output was taken as multiple different distances and then used to adjust the detection threshold to take into account the dimensions of the prototype. To avoid false activation, a 250 millisecond continuous detection requirement was added alongside a two second delay in arming the system. Also, the keypad initially output incorrect characters compared to what was actually input, which was then resolved by debugging the keypads character mapping within the Arduino sketch. While the keypad initially gave the prototype

issues, it turned out to be the most success addition to the SENTRY. Integrating the keypad allowed for the system to have multiple states of operation rather than simply designing a system that activates whenever it detects an object within its given range. This integration gives the user the opportunity to keep the SENTRY system off, turn it on whenever they want, and deactivate the alarm whenever it may be tripped. The keypad PIN integration makes the prototype more of a real security system. Finally, if there were more time allocated to this prototype development, an improvement for the SENTRY would be integrating a visual display using LCD to give the user a more detailed and comprehensive display of information about the current state of the SENTRY. Currently the UI is simply the keypad and labeled LED indicators, but a digital display would open up more opportunities for the user experience.

Appendix:

A.Photographs of SENTRY



Figure 1. Protected area view of SENTRY



Figure 2. Top view of protected area

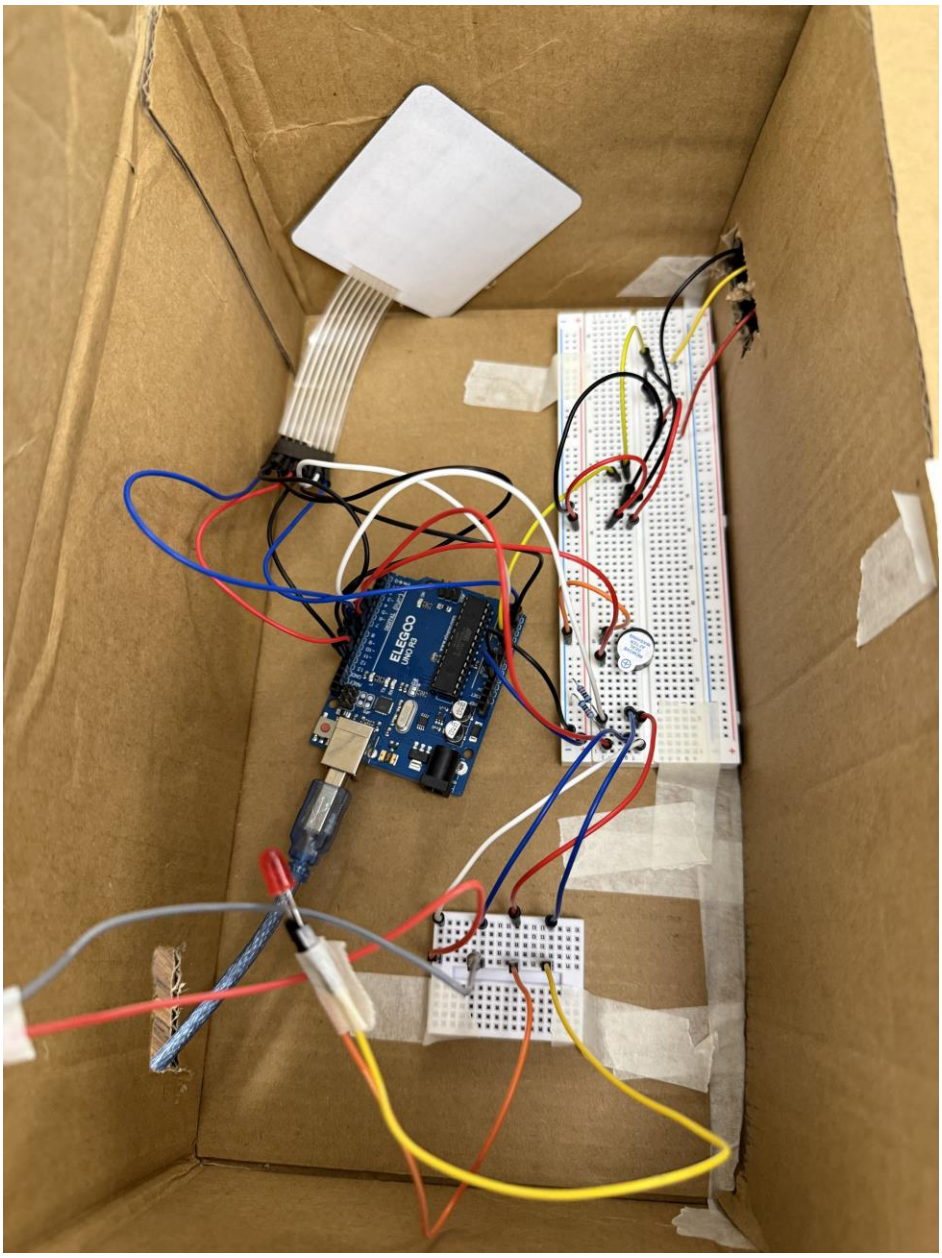


Figure 3. SENTRY hardware/circuitry

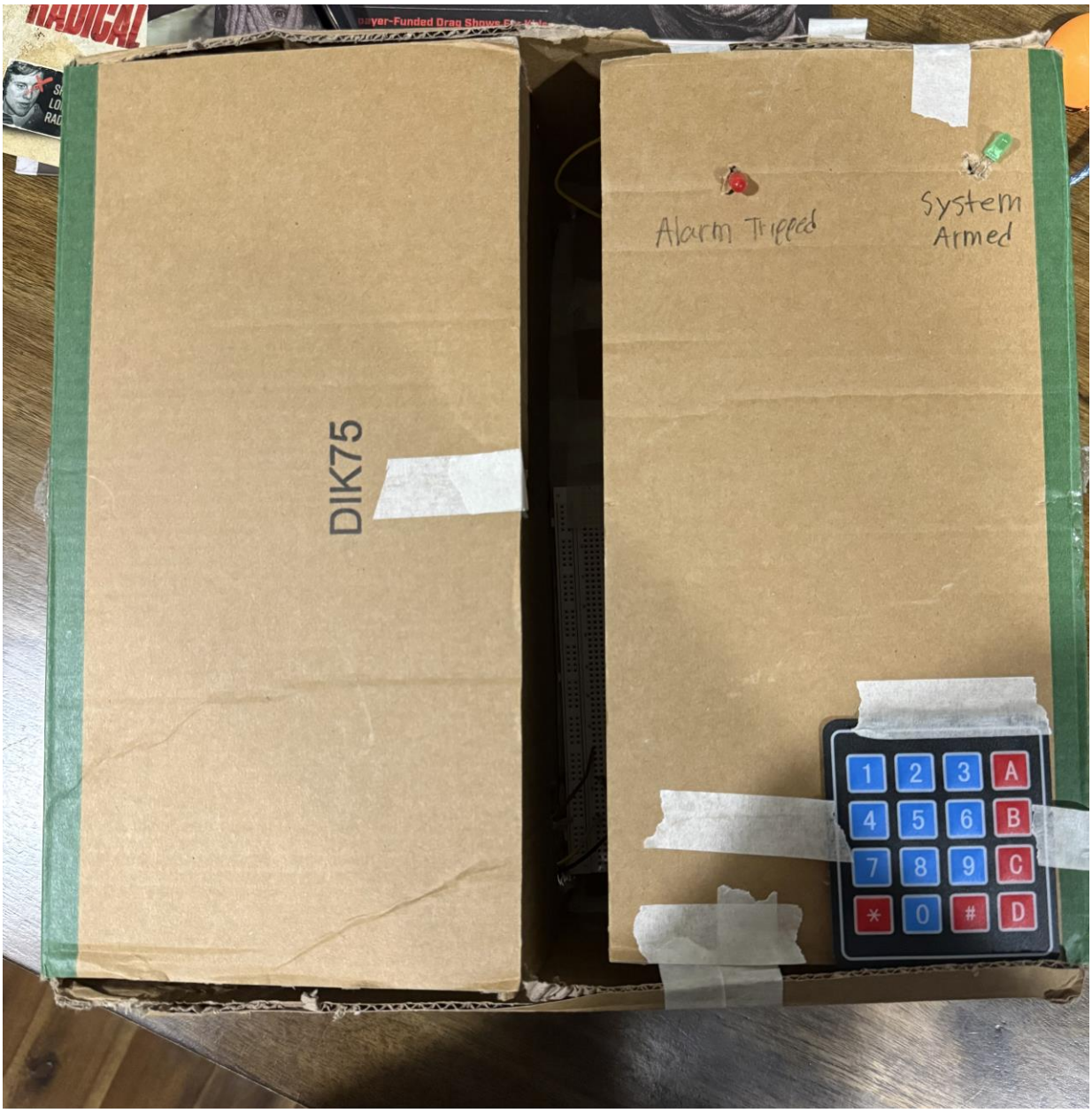


Figure 4. SENTRY user interface

B.Sentry Circuit Schematic

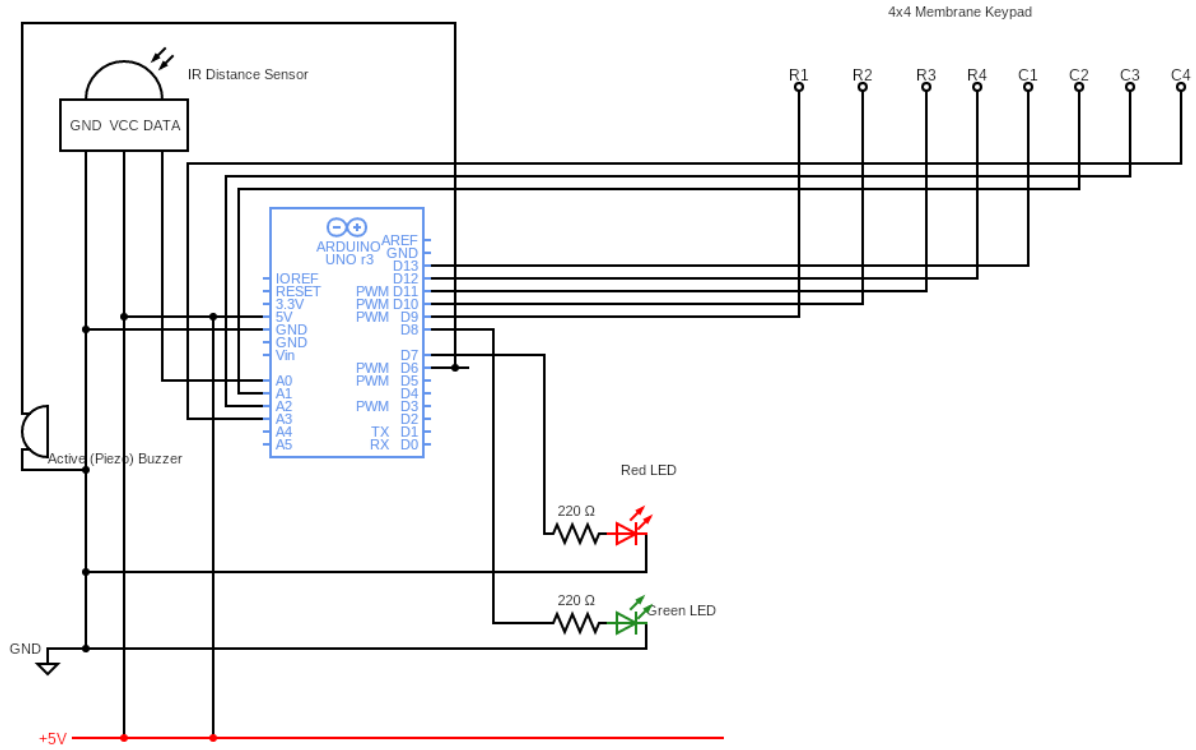


Figure 5. SENTRY Circuit Schematic

C.Arduino Sketches and Libraries

```
#include <Keypad.h> //Keypad library



// Assinged pins for components
const int irSensor = A0;
const int buzzer = 6;
const int redLED = 7;
const int greenLED = 8;

// Based on test IR distance measurements:
// Clear space = ~3-8
// 8 inches = 95-105 reading
// 6 inches = ~110-115 reading
// 4 inches = ~140-150 reading
const int triggerThreshold = 95;

// Setup the Keypad
const byte ROWS = 4;
```

```
const byte COLS = 4;

// Correct mapping for my keypad(original setup output different values than what was pressed)
char keys[ROWS][COLS] = {
{'D','C','B','A'},
{'#','9','6','3'},
{'0','8','5','2'},
{'*','7','4','1'}
};
//4 by 4 membrane keypad
byte rowPins[ROWS] = {9, 10, 11, 12};
byte colPins[COLS] = {13, A1, A2, A3};

Keypad keypad = Keypad(
makeKeymap(keys),
rowPins,
colPins,
ROWS,
COLS
);

// Stores correct pin(password); stores user inputed code
String password = "1234";
String enteredCode = "";

//Observes if the system is armed or if the trip wire has been tripped
//and set off the alarm.
bool armed = false;
bool alarmTriggered = false;

//Variables for control system for flashing red LED alarm indicator
unsigned long previousFlashMillis = 0;
const long flashInterval = 400;
bool redLEDState = false;

// Object(person) must be detected for 250 ms
// before alarm the activates and sounds alarm and flashes
// indicator. Basically a continuous IR dection is needed before alarm is tripped.
unsigned long detectionStart = 0;
const long detectionDelay = 250;
bool detectingObject = false;
unsigned long armedStartTime = 0;
const long armingDelay = 2000;



// User-defined function to determine if the IR sensor detects an object
```

```
bool objectDetected(int sensorValue)
{
if (sensorValue >= triggerThreshold)
{
return true;
}
else
{
return false;
}
}

// User-defined function: This will arm the security system
void armSystem()
{
armed = true;
detectingObject = false;
armedStartTime = millis();
digitalWrite(greenLED, HIGH);
digitalWrite(redLED, LOW);
digitalWrite(buzzer, LOW);
Serial.println();
Serial.println("SYSTEM ARMED");
}
// User-defined function: This will disarm the security system
void disarmSystem()
{
armed = false;
alarmTriggered = false;
detectingObject = false;
digitalWrite(greenLED, LOW);
digitalWrite(redLED, LOW);
digitalWrite(buzzer, LOW);
redLEDState = false;
Serial.println();
Serial.println("SYSTEM DISARMED");
}
// User-defined function: This will be used to activate the alarm
void triggerAlarm()
{
alarmTriggered = true;
detectingObject = false;
digitalWrite(greenLED, LOW);
digitalWrite(buzzer, HIGH);
Serial.println();
Serial.println("INTRUDER DETECTED!");
Serial.println("Please enter PIN to reset alarm.");
```

```
}
// General Setup
void setup()
{
//Alarm and sensor components are set as pin modes
pinMode(irSensor, INPUT);
pinMode(buzzer, OUTPUT);
pinMode(redLED, OUTPUT);
pinMode(greenLED, OUTPUT);
// Starts with the system disarmed as if you were leaving the house, thus all outputs must be off
digitalWrite(buzzer, LOW);
digitalWrite(redLED, LOW);
digitalWrite(greenLED, LOW);
//Serial monitor will display state of the security system initially
Serial.begin(9600);
Serial.println("SECURITY SYSTEM STATE");
Serial.println("System DISARMED");
Serial.println("Please enter PIN to arm.");
}
//Loop setup
void loop()
{
// Perpetually read IR distance sensor
int sensorValue = analogRead(irSensor);
// now read keypad
char key = keypad.getKey(); // checks for pressed key on keypad

if (key)
{
// Hides the actual PIN in Serial Monitor to mimic if there was a screen on the security panel
//so that nobody can see your code
Serial.print("*");

enteredCode += key;

// Check for the PIN post 4 key password has been input by user
if (enteredCode.length() == 4)
{
if (enteredCode == password)
{
// If correct pin is input by user, system resets, and will disarm the currently active alarm
if (alarmTriggered)
{
disarmSystem();
Serial.println("ALARM RESET");
}
// Entering the correct 4 digit pin will disarm the system in the event that it is currently armed
```

```
else if (armed)
{
disarmSystem();
}
// Entering the correct pin will arm the system if it is currently disarmed
else
{
armSystem();
}
}
// If incorrect PIN is input, the system will reject it
else
{
Serial.println();
Serial.println("INCORRECT PIN TRY AGAIN");
}


// Clears the incorrect PIN so user may try again
enteredCode = "";
}
}
// IR system is only monitored when the system is active and armed
if (armed && !alarmTriggered && millis() - armedStartTime >= armingDelay)
{
// If object(person) enters the protected area, the timer starts for alloted time needed to trip alarm
if (objectDetected(sensorValue))
{
// Starts the IR detection clock
if (!detectingObject)
{
detectingObject = true;
detectionStart = millis();
}
// If the object or persons remains in the protected are for a detected time of 250 ms, then alarm will be triggered.
if (millis() - detectionStart >= detectionDelay)
{
triggerAlarm();
}
}
// Finally, if the object or person is removed before 250 ms has passed, thenthe detection timer will be reset
else
{
detectingObject = false;
}
}
else
```

```
{
detectingObject = false;
}
// Red LED(Tripped alarm visual indicator) will flash while the alarm has been tripped
if (alarmTriggered)
{
unsigned long currentMillis = millis();
if (currentMillis - previousFlashMillis >= flashInterval)
{
previousFlashMillis = currentMillis;
redLEDState = !redLEDState;
digitalWrite(redLED, redLEDState);
}
}
// Red LED will remain off if there is no active alarm
else
{
digitalWrite(redLED, LOW);
redLEDState = false;
}
}
```

D.Prototype Video

All access link to functionality video for SENTRY

IMG_7385.MOV

E.Libraries

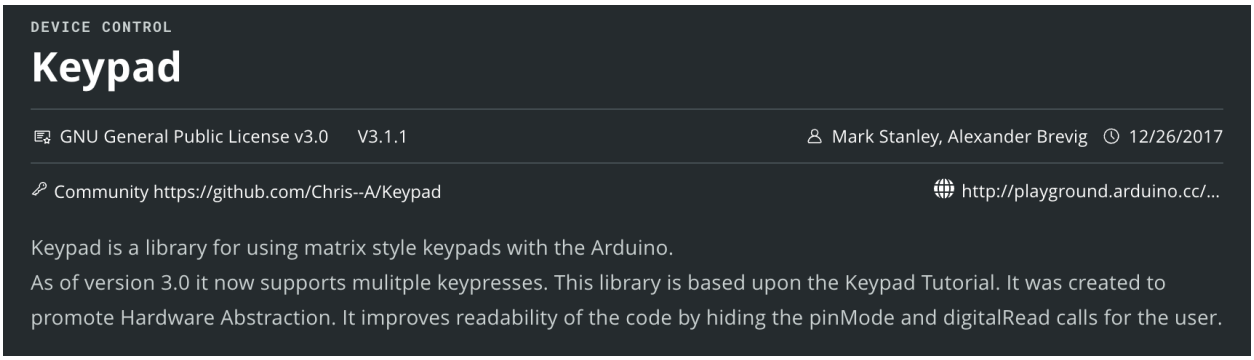


Figure 6. Used Keypad library